Operating Systems

UE23CS242B

4th Semester, Academic Year 2023

Date: 17/03/2025

Name: Roshini Ramesh	SRN: PES1UG23CS488	Section: H

EXECUTE A PROGRAM THAT WILL CREATE MULTIPLE CHILD PROCESSES IN WHICH MEMORY IS ALLOCATED DYNAMICALLY. WHILE THIS TASK IS EXECUTING, GENERATE AN OUTPUT WITH THE PROCESS ID AND THE MEMORY MAP OF EACH CHILD PROCESS IN A TREE STRUCTURE.

CODE:

My_kernel_module.c

~/Documents/my_kernel_module.c - Mousepad

File Edit Search View Document Help

my_kernel_module.c | parent_process.c | Makefile

```c
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/sched/signal.h>
#include <linux/pid.h>
#include <linux/string.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Hoshini");
MODULE_DESCRIPTION("Kernel Module to Display Process Tree Information");

static pid_t target_pid = -1;
module_param(target_pid, int, 0644);
MODULE_PARM_DESC(target_pid, "Target Process ID");

struct pid_index_map {
    pid_t parent_pid;
    int parent_index;
    pid_t child_pids[10];
    int child_indices[10];
    int num_children;
};

static struct pid_index_map parent_map[10];
static int num_parents = 0;

static void collect_process_info(struct task_struct *task, int depth)
{
    struct task_struct *child;
    struct list_head *list;

    if (!task) return;

    if (depth == 1) {
        parent_map[num_parents].parent_pid = task->pid;
        parent_map[num_parents].parent_index = num_parents;
        parent_map[num_parents].num_children = 0;
        num_parents++;

        if (num_parents >= 10) {
            printk(KERN_WARNING "Too many parents, some won't be tracked\n");
            num_parents = 0;
        }
    }
    else if (depth == 2) {
        int i;
        for (i = 0; i < num_parents; i++) {
            struct task_struct *parent = pid_task(find_get_pid(parent_map[i].parent_pid), PIDTYPE_PID);
            if (parent && (parent->pid == task->real_parent->pid)) {
                int child_idx = parent_map[i].num_children;
                parent_map[i].child_pids[child_idx] = task->pid;
                parent_map[i].child_indices[child_idx] = child_idx;
                parent_map[i].num_children++;
                break;
            }
        }
    }

    list_for_each(list, &task->children) {
        child = list_entry(list, struct task_struct, sibling);
        collect_process_info(child, depth + 1);
    }
}

static int get_parent_index(pid_t pid)
{
    int i;
    for (i = 0; i < num_parents; i++) {
        if (parent_map[i].parent_pid == pid)
            return parent_map[i].parent_index;
    }
    return -1;
}

static int get_child_index(pid_t parent_pid, pid_t child_pid)
{
    int i, j;
    for (i = 0; i < num_parents; i++) {
        if (parent_map[i].parent_pid == parent_pid) {
            for (j = 0; j < parent_map[i].num_children; j++) {
                if (parent_map[i].child_pids[j] == child_pid)
                    return parent_map[i].child_indices[j];
            }
        }
    }
    return -1;
}

static void print_task_info(struct task_struct *task, int depth)
{
    struct task_struct *child;
    struct list_head *list;
    char indent[256] = "";
    int i, parent_idx, child_idx;

    if (!task) return;

    if (depth == 0) {
        printk(KERN_INFO "Grandparent [%d]", task->pid);
    } else if (depth == 1) {
        parent_idx = get_parent_index(task->pid);
        if (parent_idx == -1) {
            printk(KERN_INFO "| - Parent Process %d [%d]", parent_idx, task->pid);
        } else {
            printk(KERN_INFO "| - Parent Process [%d]", task->pid);
        }
    } else {
        for (i = 0; i < 2; i++) {
            strcat(indent, "  ");
        }

        parent_idx = get_parent_index(task->real_parent->pid);
        child_idx = get_child_index(task->real_parent->pid, task->pid);

        if (parent_idx != -1 && child_idx != -1) {
            printk(KERN_INFO "%s| - - Child %d [%d]",
                   indent, child_idx, task->pid);
        } else {
            printk(KERN_INFO "%s| - - Child [%d]", indent, task->pid);
        }
    }

    list_for_each(list, &task->children) {
        child = list_entry(list, struct task_struct, sibling);
        print_task_info(child, depth + 1);
    }
}

static int __init process_tree_init(void)
{
    struct task_struct *task = NULL;

    printk(KERN_INFO "Process tree module loading for PID: %d\n", target_pid);

    task = pid_task(find_get_pid(target_pid), PIDTYPE_PID);

    if (!task) {
        printk(KERN_WARNING "Process with PID %d not found\n", target_pid);
        return -ESRCH;
    }

    memset(parent_map, 0, sizeof(parent_map));
    num_parents = 0;
    collect_process_info(task, 0);

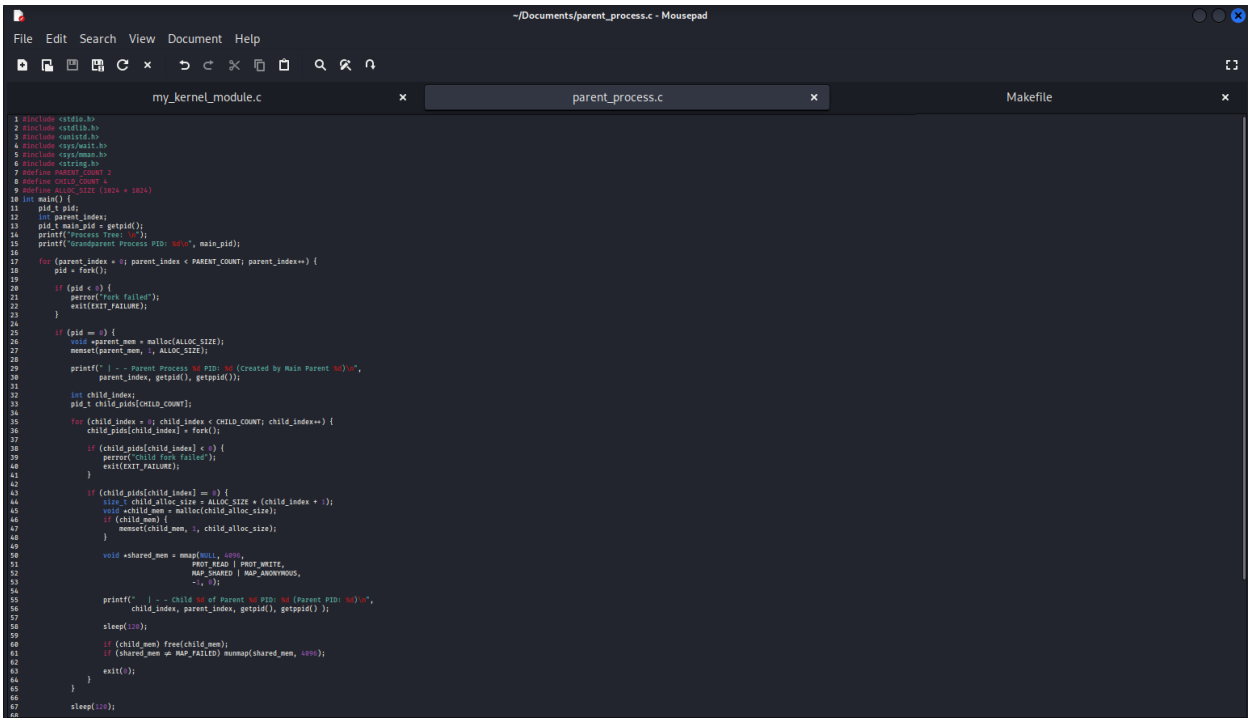
    printk(KERN_INFO "Process Tree:");
    print_task_info(task, 0);

    return 0;
}

static void __exit process_tree_exit(void)
{
    printk(KERN_INFO "Process tree module unloaded\n");
}

module_init(process_tree_init);
module_exit(process_tree_exit);
```

Parent_process.c





Makefile



OUTPUT:

Output of parent_process:

```
┌──(roshini㉿vbox)-[~/Documents]
└─$ ./parent_process
Process Tree:
Grandparent Process PID: 29881
 | - - Parent Process 0 PID: 29882 (Created by Main Parent 29881)
 | - - Parent Process 1 PID: 29883 (Created by Main Parent 29881)
   | - - Child 0 of Parent 0 PID: 29884 (Parent PID: 29882)
   | - - Child 0 of Parent 1 PID: 29885 (Parent PID: 29883)
   | - - Child 1 of Parent 0 PID: 29886 (Parent PID: 29882)
   | - - Child 1 of Parent 1 PID: 29887 (Parent PID: 29883)
   | - - Child 2 of Parent 0 PID: 29888 (Parent PID: 29882)
   | - - Child 2 of Parent 1 PID: 29889 (Parent PID: 29883)
   | - - Child 3 of Parent 1 PID: 29891 (Parent PID: 29883)
   | - - Child 3 of Parent 0 PID: 29890 (Parent PID: 29882)
```

Output of dmesg after running sudo insmod target_id=29881:

```
[ 5320.249437] Process tree module loading for PID: 29881
[ 5320.249446] Process Tree:
[ 5320.249446] Grandparent [29881]
[ 5320.249447] | - Parent Process 0 [29882]
[ 5320.249448]     | - - Child 0 [29884]
[ 5320.249450]     | - - Child 1 [29886]
[ 5320.249450]     | - - Child 2 [29888]
[ 5320.249451]     | - - Child 3 [29890]
[ 5320.249505] | - Parent Process 1 [29883]
[ 5320.249505]     | - - Child 0 [29885]
[ 5320.249506]     | - - Child 1 [29887]
[ 5320.249507]     | - - Child 2 [29889]
[ 5320.249507]     | - - Child 3 [29891]
[ 5328.062204] Process tree module unloaded
[ 5330.330519] Process tree module loading for PID: 29881
```